Documentación de mecánicas

Proyecto Moonveil – Juego 2D en Unity 6

Este documento describe las mecánicas implementadas hasta ahora: qué hace cada script, cómo se configura en Unity y cómo se relacionan entre sí.

Contenido

- 1. Movimiento del jugador (**PlayerMovement**)
- 2. Sprint con estamina (**PlayerSprint**)
- 3. Cámara con seguimiento y zoom (**CameraFollow**)
- 4. Animación del personaje (**PlayerAnimacion**)
- 5. Barra de estamina sobre el jugador (**BarraEstamina**)
- 6. Colisiones con el mapa (Tilemaps)
- 7. Resumen de dependencias entre scripts

Requisitos generales

- Unity 6 con el paquete **Input System** activo.
- El objeto **player** tiene: Rigidbody2D, Collider2D, Sprite Renderer y los scripts PlayerMovement, PlayerSprint y PlayerAnimacion.
- Solo se contempla teclado y ratón (no hay mando).

1. Movimiento del jugador

Script: PlayerMovement.cs **Objeto:** player

Mueve al jugador en 8 direcciones con las teclas **WASD** o las **flechas**. Lee la entrada en `Update` y aplica la velocidad al Rigidbody2D en `FixedUpdate`, que es lo correcto para físicas.

Cómo funciona

- Se crea una **InputAction** con dos bindings de tipo 2DVector (WASD y flechas).
- En `OnEnable`/`OnDisable` se activa y desactiva la acción; las InputActions creadas por código no funcionan si no se activan.
- El vector de entrada se **normaliza** para que ir en diagonal no sea más rápido.
- La propiedad pública `Velocidad` permite que PlayerSprint cambie la velocidad.

Parámetros del Inspector

Parámetro	Valor	Descripción
Velocidad	6	Unidades por segundo al caminar.

Código

```
using UnityEngine;
using UnityEngine.InputSystem;

[RequireComponent(typeof(Rigidbody2D))]
public class PlayerMovement : MonoBehaviour
{
    [SerializeField] private float velocidad = 6f;

    private Rigidbody2D rb;
    private Vector2 inputMovimiento;
    private InputAction accionMovimiento;

    private void Awake()
    {
        rb = GetComponent<Rigidbody2D>();

        accionMovimiento = new InputAction("Movimiento", InputActionType.Value);
        accionMovimiento.AddCompositeBinding("2DVector")
            .With("Up", "<Keyboard>/w")
            .With("Down", "<Keyboard>/s")
            .With("Left", "<Keyboard>/a")
            .With("Right", "<Keyboard>/d");
        accionMovimiento.AddCompositeBinding("2DVector")
            .With("Up", "<Keyboard>/upArrow")
            .With("Down", "<Keyboard>/downArrow")
            .With("Left", "<Keyboard>/leftArrow")
            .With("Right", "<Keyboard>/rightArrow");
    }

    private void OnEnable() => accionMovimiento.Enable();
    private void OnDisable() => accionMovimiento.Disable();

    private void Update()
    {
        inputMovimiento = accionMovimiento.ReadValue<Vector2>();
    }

    private void FixedUpdate()
    {
        rb.linearVelocity = inputMovimiento.normalized * velocidad;
    }

    public float Velocidad
    {
        get => velocidad;
        set => velocidad = value;
    }
}
```

2. Sprint con estamina

Script: PlayerSprint.cs **Objeto:** player (junto a PlayerMovement)

Mientras se mantiene **Shift izquierdo** el jugador corre más rápido y gasta estamina. Al soltar la tecla o quedarse sin estamina, el sprint se detiene; tras una espera la estamina se recarga sola.

Cómo funciona

- Shift pulsado **y** estamina > 0: se activa `esprintando`, se resta estamina cada segundo y se multiplica la velocidad base.
- En caso contrario: se restaura la velocidad base y empieza una cuenta atrás de espera.
- Cuando la espera llega a 0, la estamina se recarga hasta el máximo.
- Cada vez que se esprinta, la espera se reinicia (no se recarga mientras corres).
- `esprintando` es público: lo leen la cámara y la animación.
- `PorcentajeEstamina()` devuelve un valor de 0 a 1 para la barra de UI.

Parámetros del Inspector

Parámetro	Valor	Descripción
Multiplicador Sprint	1.6	Velocidad al esprintar = velocidad base × este valor.
Estamina Maxima	30	Capacidad total de estamina.
Velocidad Gasto	10	Estamina que se gasta por segundo esprintando (30 / 10 = 3 s de sprint continuo).
Tiempo Espera Recarga	5	Segundos sin esprintar antes de empezar a recargar.
Velocidad Recarga	10	Estamina que se recupera por segundo.

Código

```
using UnityEngine;
using UnityEngine.InputSystem;

[RequireComponent(typeof(PlayerMovement))]
public class PlayerSprint : MonoBehaviour
{
    [SerializeField] private float multiplicadorSprint = 1.6f;
    [SerializeField] private float estaminaMaxima = 30f;
    [SerializeField] private float velocidadGasto = 10f;
    [SerializeField] private float tiempoEsperaRecarga = 5f;
    [SerializeField] private float velocidadRecarga = 10f;

    private PlayerMovement movimiento;
    private InputAction accionSprint;

    private float velocidadBase;
    private float estaminaActual;
    private float timerEspera;
    public bool esprintando = false;

    private void Awake()
    {
        movimiento = GetComponent<PlayerMovement>();

        accionSprint = new InputAction("Sprint", InputActionType.Button);
        accionSprint.AddBinding("<Keyboard>/leftShift");
    }

    private void OnEnable() => accionSprint.Enable();
    private void OnDisable() => accionSprint.Disable();

    private void Start()
    {
        velocidadBase = movimiento.Velocidad;
        estaminaActual = estaminaMaxima;
    }
```

```
    private void Update()
    {
        bool shiftPulsado = accionSprint.IsPressed();

        if (shiftPulsado && estaminaActual > 0f)
        {
            esprintando = true;
            estaminaActual -= velocidadGasto * Time.deltaTime;
            timerEspera = tiempoEsperaRecarga;
            movimiento.Velocidad = velocidadBase * multiplicadorSprint;

            if (estaminaActual < 0f) estaminaActual = 0f;
        }
        else
        {
            esprintando = false;
            movimiento.Velocidad = velocidadBase;

            if (timerEspera > 0f)
            {
                timerEspera -= Time.deltaTime;
            }
            else
            {
                estaminaActual += velocidadRecarga * Time.deltaTime;
                if (estaminaActual > estaminaMaxima) estaminaActual = estaminaMaxima;
            }
        }
    }

    public float PorcentajeEstamina()
    {
        return estaminaActual / estaminaMaxima;
    }
}
```

3. Cámara con seguimiento y zoom

Script: CameraFollow.cs **Objeto:** Main Camera

La cámara sigue al jugador de forma suave y, al esprintar, hace un ligero **acercamiento** (zoom). Al dejar de esprintar vuelve al tamaño normal.

Cómo funciona

- `Vector3.SmoothDamp` mueve la cámara hacia el jugador con suavidad.
- El zoom 2D se hace cambiando `orthographicSize`: **menor = más cerca**, mayor = más lejos.
- Si el jugador esprinta, el objetivo es *tamaño sprint* (más pequeño); si no, *tamaño normal*.
- `Mathf.MoveTowards` avanza hacia el objetivo sin pasarse, evitando que la cámara rebote.
- En `Awake` se iguala el tamaño inicial al normal para que no haya zoom al pulsar Play.
- Lee `esprintando` de PlayerSprint.

Parámetros del Inspector

Parámetro	Valor	Descripción
Objetivo	player	Transform al que sigue la cámara.
Suavizado Posicion	0.15	Tiempo de suavizado del seguimiento.
Offset	(0, 0, -10)	Desfase respecto al jugador (la Z debe ser negativa en 2D).
Tamaño Normal	6.5	orthographicSize habitual (cámara más alejada).
Tamaño Sprint	5	orthographicSize al esprintar (cámara más cerca).
Velocidad Zoom	3	Rapidez de la transición del zoom.

Los valores de tamaño son los comentados durante el desarrollo; ajústalos en el Inspector a tu gusto.

Código

```
using UnityEngine;

public class CameraFollow : MonoBehaviour
{
    [SerializeField] private Transform objetivo;
    [SerializeField] private float suavizadoPosicion = 0.15f;
    [SerializeField] private Vector3 offset = new Vector3(0f, 0f, -10f);

    [SerializeField] private float tamañoNormal = 6.5f;
    [SerializeField] private float tamañoSprint = 5f;
    [SerializeField] private float velocidadZoom = 3f;

    private Camera cam;
    private PlayerSprint sprint;
    private Vector3 velocidad = Vector3.zero;

    private void Awake()
    {
        cam = GetComponent<Camera>();
        sprint = Object.FindAnyObjectByType<PlayerSprint>();
        cam.orthographicSize = tamañoNormal;
    }

    private void LateUpdate()
    {
        if (objetivo == null) return;

        // Movimiento
        Vector3 posicionObjetivo = objetivo.position + offset;
        transform.position = Vector3.SmoothDamp(transform.position, posicionObjetivo,
                                                ref velocidad, suavizadoPosicion);

        // Zoom
        float tamañoObjetivo;

        if (sprint.esprintando)
```

```
            tamañoObjetivo = tamañoSprint;
        else
            tamañoObjetivo = tamañoNormal;

        cam.orthographicSize = Mathf.MoveTowards(cam.orthographicSize, tamañoObjetivo,
                                                 velocidadZoom * Time.deltaTime);
    }
}
```

4. Animación del personaje

Script: PlayerAnimacion.cs **Objeto:** player

Cambia el sprite del personaje según la dirección de movimiento y acelera la animación al esprintar. No usa Animator: alterna los sprites directamente desde código.

Cómo funciona

- Hay tres listas de sprites (abajo, arriba, lado), con 3 sprites cada una: `jugador_abajo_0/1/2`, etc.
- La dirección se decide con la velocidad del Rigidbody2D: si domina el eje X se usa el lado, si domina Y, arriba o abajo.
- Solo existen sprites de lado en una dirección; para la otra se usa `flipX`.
- Parado: se muestra el sprite 0 de la dirección actual.
- Andando: se alterna entre los sprites 1 y 2 cada cierto tiempo.
- Al esprintar se usa un tiempo entre frames menor, así las piernas se mueven más rápido.

Configuración en Unity

- Añadir el script al **player**.
- Arrastrar los 3 sprites de cada dirección a su lista, en orden 0, 1, 2.
- Si al ir a la derecha el sprite mira a la izquierda, cambiar `vel.x < 0` por `vel.x > 0`.
- La **sombra** es un objeto hijo del player con su propio Sprite Renderer y un *Order in Layer* menor.

Parámetros del Inspector

Parámetro	Valor	Descripción
Sprites Abajo / Arriba / Lado	3 sprites	Frames de cada dirección.
Tiempo Caminar	0.2	Segundos entre frames al caminar.
Tiempo Sprint	0.1	Segundos entre frames al esprintar.

Código

```
using UnityEngine;

public class PlayerAnimacion : MonoBehaviour
{
    [SerializeField] private Sprite[] spritesAbajo;
    [SerializeField] private Sprite[] spritesArriba;
    [SerializeField] private Sprite[] spritesLado;

    [SerializeField] private float tiempoCaminar = 0.2f;
    [SerializeField] private float tiempoSprint = 0.1f;

    private SpriteRenderer sr;
    private Rigidbody2D rb;
    private PlayerSprint sprint;

    private Sprite[] spritesActuales;
    private float timer;
    private int frame;

    private void Awake()
    {
        sr = GetComponent<SpriteRenderer>();
        rb = GetComponent<Rigidbody2D>();
        sprint = GetComponent<PlayerSprint>();
        spritesActuales = spritesAbajo;
    }

    private void Update()
    {
        Vector2 vel = rb.linearVelocity;

        // Parado: primer sprite de la direccion actual
```

```
        if (vel == Vector2.zero)
        {
            frame = 0;
            timer = 0f;
            sr.sprite = spritesActuales[0];
            return;
        }

        // Elegir direccion
        if (Mathf.Abs(vel.x) > Mathf.Abs(vel.y))
        {
            spritesActuales = spritesLado;
            sr.flipX = vel.x < 0;
        }
        else if (vel.y > 0)
        {
            spritesActuales = spritesArriba;
            sr.flipX = false;
        }
        else
        {
            spritesActuales = spritesAbajo;
            sr.flipX = false;
        }

        // Velocidad de la animacion
        float tiempoEntreFrames;
        if (sprint.esprintando)
            tiempoEntreFrames = tiempoSprint;
        else
            tiempoEntreFrames = tiempoCaminar;

        // Cambiar de frame
        timer += Time.deltaTime;
        if (timer >= tiempoEntreFrames)
        {
            timer = 0f;
            if (frame == 1)
                frame = 2;
            else
                frame = 1;
        }

        sr.sprite = spritesActuales[frame];
    }
}
```

5. Barra de estamina

Script: BarraEstamina.cs **Objeto:** BarraFondo (UI)

Una barra pequeña que aparece **encima del jugador** mientras esprinta, se vacía al gastar estamina y desaparece al dejar de esprintar.

Jerarquía de UI

```
Canvas  (Render Mode: Screen Space - Overlay)
 +-- BarraFondo        (Image oscura + Canvas Group + script BarraEstamina)
      +-- BarraRelleno  (Image tipo Filled)
```

Configuración de BarraFondo

Campo	Valor	Motivo
Width / Height	60 x 8 (aprox.)	Barra pequeña sobre el jugador.
Pivot	(0.5, 0.5)	Centra la barra sobre el jugador (es el valor por defecto).
Canvas Group	valores por defecto	Permite ocultar la barra y sus hijos con alpha = 0.
Jugador (script)	player	Referencia a PlayerSprint.
Relleno (script)	BarraRelleno	Imagen que se va rellenando.
Offset (script)	(0, 1, 0)	Desfase sobre el jugador, en unidades del mundo. Subir Y para elevar la barra.

Configuración de BarraRelleno

Campo	Valor	Motivo
Source Image	cualquier sprite (ej. UISprite)	Sin sprite no aparece la opción Filled.
Image Type	Filled	Permite rellenar la imagen por porcentaje.
Fill Method	Horizontal	La barra se llena de lado a lado.
Fill Origin	Left	Se vacía de derecha a izquierda.
Tamaño	igual que BarraFondo	Anclaje stretch (Alt+Shift) con Left, Right, Top y Bottom a 0.

Cómo funciona

- Cada frame, el script pide a PlayerSprint su porcentaje de estamina (0 a 1) con `PorcentajeEstamina()` y lo asigna a `fillAmount`: 1 = llena, 0 = vacía.
- `Camera.WorldToScreenPoint` convierte la posición del jugador más el offset en una posición de pantalla, así la barra lo sigue.
- Si el jugador esprinta, `alpha = 1` (visible); si no, `alpha = 0` (oculta).
- Se usa Canvas Group y no `SetActive(false)` porque el script está en el mismo objeto: si se desactivara a sí mismo, dejaría de ejecutarse y nunca volvería a mostrarse.
- Si aparece un error con el EventSystem, usar el botón *Replace with InputSystemUIInputModule* en su Inspector.

Código

```
using UnityEngine;
using UnityEngine.UI;

public class BarraEstamina : MonoBehaviour
{
    [SerializeField] private PlayerSprint jugador;
    [SerializeField] private Image relleno;
```

```
    [SerializeField] private Vector3 offset = new Vector3(0f, 1f, 0f);

    private CanvasGroup grupo;
    private Camera cam;

    private void Awake()
    {
        grupo = GetComponent<CanvasGroup>();
        cam = Camera.main;
    }

    private void LateUpdate()
    {
        // Rellenar la barra
        relleno.fillAmount = jugador.PorcentajeEstamina();

        // Colocarla encima del jugador
        transform.position = cam.WorldToScreenPoint(jugador.transform.position + offset);

        // Mostrar solo al esprintar
        if (jugador.esprintando)
            grupo.alpha = 1f;
        else
            grupo.alpha = 0f;
    }
}
```

Para mostrar también la barra mientras se recarga, cambiar la condición por jugador.PorcentajeEstamina() < 1f.

6. Colisiones con el mapa

El mapa está formado por varios Tilemaps dentro de un Grid: **Tilemap** (suelo), **Casas** y **Vallas**. Solo los que deben bloquear al jugador llevan colisión.

- En **Casas** y **Vallas**: añadir *Tilemap Collider 2D* y *Composite Collider 2D*.
- En Unity 6, en el Tilemap Collider 2D poner **Geometry Type = Outlines** (ya no existe la casilla Used By Composite).
- El Rigidbody2D que se añade al Tilemap debe ser **Static**.
- El suelo no lleva collider, porque el jugador camina sobre él.
- El **player** necesita un Collider2D (por ejemplo Capsule Collider 2D) y **Freeze Rotation Z** activado en las Constraints del Rigidbody2D.

7. Resumen de dependencias entre scripts

Los scripts se comunican entre sí de esta forma:

Script	Necesita de	Qué usa
PlayerMovement	Rigidbody2D	Aplica la velocidad al jugador.
PlayerSprint	PlayerMovement	Cambia la propiedad Velocidad al esprintar.
CameraFollow	PlayerSprint	Lee esprintando para hacer zoom.
PlayerAnimacion	Rigidbody2D, PlayerSprint, SpriteRenderer	Lee la velocidad (dirección) y esprintando (rapidez).
BarraEstamina	PlayerSprint, Camera, CanvasGroup	Llama a PorcentajeEstamina(), lee esprintando y sigue al jugador.

Flujo del sistema de sprint

- El jugador mantiene **Shift** → PlayerSprint activa `esprintando` y sube la velocidad.
- PlayerMovement mueve al jugador más rápido.
- PlayerAnimacion acelera el cambio de sprites.
- CameraFollow acerca la cámara.
- BarraEstamina aparece sobre el jugador y refleja cómo baja la estamina.

Posibles mejoras futuras

- Animación distinta para correr cuando haya sprites propios (o pasar a Animator).
- Evitar volver a esprintar hasta recuperar un mínimo de estamina.